

ANSIBLE cours (Niveau débutant)

Introduction

Ansible est un outil open-source d'automatisation informatique qui permet de gérer des configurations, déployer des applications et orchestrer des tâches complexes sur plusieurs serveurs de manière simple et efficace. Il est principalement utilisé pour l'automatisation des infrastructures, le déploiement d'applications et la gestion continue des configurations.

Voici les principales caractéristiques d'Ansible :

Sans agent : Ansible n'a pas besoin d'installer de logiciel sur les machines à gérer. Il utilise SSH pour communiquer avec les serveurs cibles.

Facilité d'utilisation : Les tâches d'Ansible sont définies dans des fichiers YAML appelés playbooks, qui sont lisibles et faciles à écrire.

Idempotence : Ansible s'assure que les actions décrites dans les playbooks ne sont effectuées que si elles sont nécessaires. Si une machine est déjà dans l'état désiré, Ansible ne fait rien.

Modularité : Il est extensible via des modules, qui sont des blocs réutilisables de code pour gérer différents services et infrastructures (réseaux, bases de données, stockage, etc.).

Utilisation variée : On peut s'en servir pour gérer des serveurs, des bases de données, des applications, des réseaux, ainsi que pour orchestrer des environnements cloud, tels que AWS, Azure, et d'autres.

Ansible est souvent utilisé pour l'automatisation de masse dans des environnements hybrides (cloud et locaux), notamment dans des contextes DevOps pour simplifier les processus de CI/CD.

Mon objectif est de fournir des solutions d'infrastructure et de déploiement fiables, évolutives et sécurisées pour aider les entreprises à atteindre leurs objectifs commerciaux.

rambeloson.boni9@gmail.com

Explication de l'idempotent

L'idempotence en Ansible signifie que lorsque vous exécutez un playbook, Ansible va vérifier l'état actuel de la machine cible avant de faire quoi que ce soit. Si la machine est déjà dans l'état souhaité, aucune action ne sera effectuée. Cela permet d'éviter la répétition de tâches inutiles et de s'assurer que l'état du système est maintenu de manière cohérente, même si le playbook est exécuté plusieurs fois.

Prenons un exemple simple : vous voulez vous assurer qu'un package est installé sur une machine.

Exemple :

Voici un `playbook` Ansible qui garantit que le package `nginx` est installé :

```
- hosts: webserver
  tasks:
    - name: Assurez-vous que nginx est installé
      apt:
        name: nginx
        state: present
```

Première exécution : Si `nginx` n'est pas installé, Ansible va l'installer.

Exécutions suivantes : Si `nginx` est déjà installé, Ansible ne fera rien, car l'état souhaité (le package installé) est déjà atteint.

Ansible fait cette vérification d'état avant d'exécuter chaque tâche. Cela signifie qu'il ne change un composant que s'il détecte qu'il ne correspond pas à l'état souhaité. C'est ce qu'on appelle l'idempotence. Cela évite de réinstaller un logiciel ou de redémarrer un service inutilement.

Lancement du Playbook

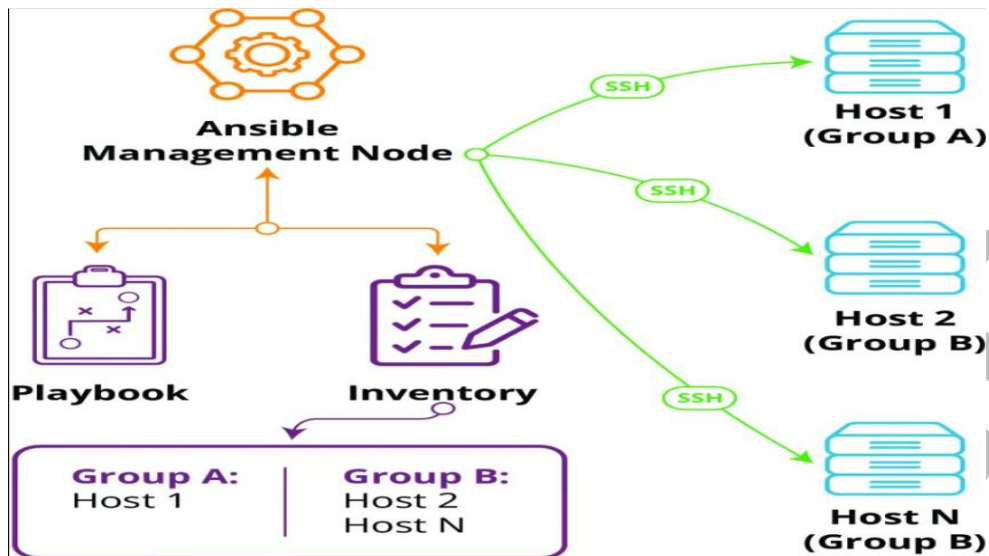
Ansible communique avec des machines distantes via le protocole SSH. Il faut donc vous assurer que vous pouvez vous connecter à vos différents nœuds distants de votre inventaire en utilisant le protocole SSH. Dans notre cas nous utiliserons la commande `ssh-copy-id` afin d'ajouter la clé SSH publique du serveur de contrôle au fichier `authorized_keys` sur nos systèmes gérés.

```
ssh-copy-id -i ~/.ssh/id_rsa.pub user@[adresse_ip_serveur]
```

Mon objectif est de fournir des solutions d'infrastructure et de déploiement fiables, évolutives et sécurisées pour aider les entreprises à atteindre leurs objectifs commerciaux.

rambeloson.boni9@gmail.com

Architecture Fonctionnelle



À la base, Ansible repose sur une architecture simple mais puissante, principalement composée de deux types de machines : le nœud de contrôle et les nœuds gérés.

Nœud de contrôle : la machine sur laquelle Ansible est installé et à partir de laquelle toutes les tâches et tous les playbooks sont exécutés. Vous pouvez avoir plusieurs nœuds de contrôle.

Nœuds gérés : les serveurs (nœuds) que vous gérez avec Ansible. Il n'est pas nécessaire d'installer Ansible sur ces nœuds, car le nœud de contrôle communique avec eux via SSH ou PowerShell.

Playbook Ansible

Les playbooks Ansible servent à orchestrer des processus informatiques. Un playbook est un fichier YAML (avec une extension **.yml** ou **.yaml**) qui contient un ou plusieurs plays (jeux de tâches) et qui permet de définir l'état souhaité d'un système, tandis qu'un module Ansible est un script autonome qui peut être utilisé à l'intérieur d'un playbook Ansible.

Mon objectif est de fournir des solutions d'infrastructure et de déploiement fiables, évolutives et sécurisées pour aider les entreprises à atteindre leurs objectifs commerciaux.

rambeloson.boni9@gmail.com

Résumé par Mr Bonitah RAMBELOSON
Consultant DevOps/SysOps | Ingénieur de production IT

Les playsbook consistent en une série de tâches à exécuter sur les hôtes sélectionnés dans le fichier d'inventaire Ansible. Les tâches sont les éléments qui constituent un play et qui appellent les modules Ansible. Dans un playbook, les tâches sont exécutées dans l'ordre de déclaration.

Pendant son exécution, Ansible suit l'état du système. S'il analyse un système et détecte une incohérence entre la description du système dans le playbook et son état réel, il fait les ajustements nécessaires pour que l'état réel corresponde à celui du playbook.

```
---
##### WEB SERVER #####

- hosts: web
  become: true

  tasks:
    - name: install apache and php last version
      apt:
        name:
          - apache2
          - php
          - php-mysql
        state: present
        update_cache: yes

    - name: Give writable mode to http folder
      file:
        path: /var/www/html
        state: directory
        mode: '0755'

##### DATABASE SERVER #####

- hosts: db
  become: true

  tasks:
    - name: install mysql
      apt:
        name:
          - mysql-server
          - python-mysqldb # for mysql_db and mysql_user modules
        state: present
        update_cache: yes
```

Mon objectif est de fournir des solutions d'infrastructure et de déploiement fiables, évolutives et sécurisées pour aider les entreprises à atteindre leurs objectifs commerciaux.

rambeloson.boni9@gmail.com

Explication du Playbook

Dans un Playbook, vous pouvez choisir les machines à cibler (**déclarer dans un fichier inventory**) pour effectuer vos tâches. Dans notre cas on commence par cibler notre serveur web :

```
---  
- hosts: web
```

Fichier inventory.ini

```
[web]  
IP_WEB  
  
[db]  
IP_DB
```

Élévation de privilèges veut dire qu'on demande au moteur Ansible d'exécuter toutes nos tâches en tant qu'utilisateur root grâce au mot-clé **become** :

```
become: true
```

Les tâches dans un Playbook Ansible

Chaque hôte (**web et db**) contient une liste de tâches au-dessous du mot-clé tasks. Les tâches sont exécutées dans l'ordre, une à la fois, sur toutes les machines correspondant au modèle d'hôte avant de passer à la tâche suivante.

Le but de chaque tâche est d'exécuter un module Ansible avec des arguments très spécifiques.

Chaque tâche peut débuter avec le mot-clé **name**, qui est simplement une brève description de votre tâche. Cette information s'affichera à la sortie de l'exécution du Playbook, son but principal est de pouvoir distinguer et décrire vos différentes tâches. Il est donc utile de fournir

Mon objectif est de fournir des solutions d'infrastructure et de déploiement fiables, évolutives et sécurisées pour aider les entreprises à atteindre leurs objectifs commerciaux.

rambeloson.boni9@gmail.com

Résumé par Mr Bonitah RAMBELOSON
Consultant DevOps/SysOps | Ingénieur de production IT

de bonnes petites descriptions pour chaque tâche. Si le champ n'est pas saisi alors le nom du module sera utilisée comme sorties. Au-dessous du mot-clé **name**, vous insérez le nom du module avec ses différents paramètres.

Dans notre projet, notre premier besoin consiste à installer les packages apache2, php et php-mysql avec le gestionnaire de paquets apt.

Maintenant l'étape suivante consiste à rechercher les différents paramètres que propose le module apt. Dans notre cas on cherche à installer la dernière version des packages apache2, php et php-mysql. En lisant la documentation on peut vite s'apercevoir qu'il existe les options suivantes :

name (type: liste) : liste de noms de packages (on peut aussi spécifier la version du package ex curl=1.6 ou curl=1.0*).

state (type: string) : indique l'état du package, voici un exemple des valeurs possibles :

latest : assure que c'est toujours la dernière version qui est installée.

present : vérifie si le package est déjà installé, si c'est le cas il ne fait rien, sinon il l'installe.

absent : supprime le package s'il est déjà installé.

update_cache (type: booléen) : exécute l'équivalent de la commande apt-get update avant l'installation des paquets.

La commande pour lancer votre playbook

ansible-playbook **playbook.yml**

Mon objectif est de fournir des solutions d'infrastructure et de déploiement fiables, évolutives et sécurisées pour aider les entreprises à atteindre leurs objectifs commerciaux.

rambeloson.boni9@gmail.com